

[Algorithm]

트리 활용 - 힙 정렬

인하대학교 소프트웨어융합공학과

김종훈 교수



이진힙

- 이진힙(Binary Heap)은 우선순위큐(Priority Queue)를 구현하는 가장 기본적인 자료구조이다. 힙 조건을 만족하는 완전 이진트리 (Complete Binary Tree)이다.
- 우선순위큐(Priority Queue)
 - 가장 높은 우선순위를 가진 항목에 접근, 삭제와 임의의 우선순위를 가진 항목을 삽입을 지원하는 자료구조

이진힙

□ 스택이나 큐도 일종의 우선순위큐

- **스택**: 가장 마지막으로 삽입된 항목이 가장 높은 우선순위를 가지므로, 최근 시간일수록 높은 우선순위를 부여
- **큐**: 먼저 삽입된 항목이 우선순위가 더 높다. 따라서 이른 시간일수록 더 높은 우선순위를 부여

이진힙

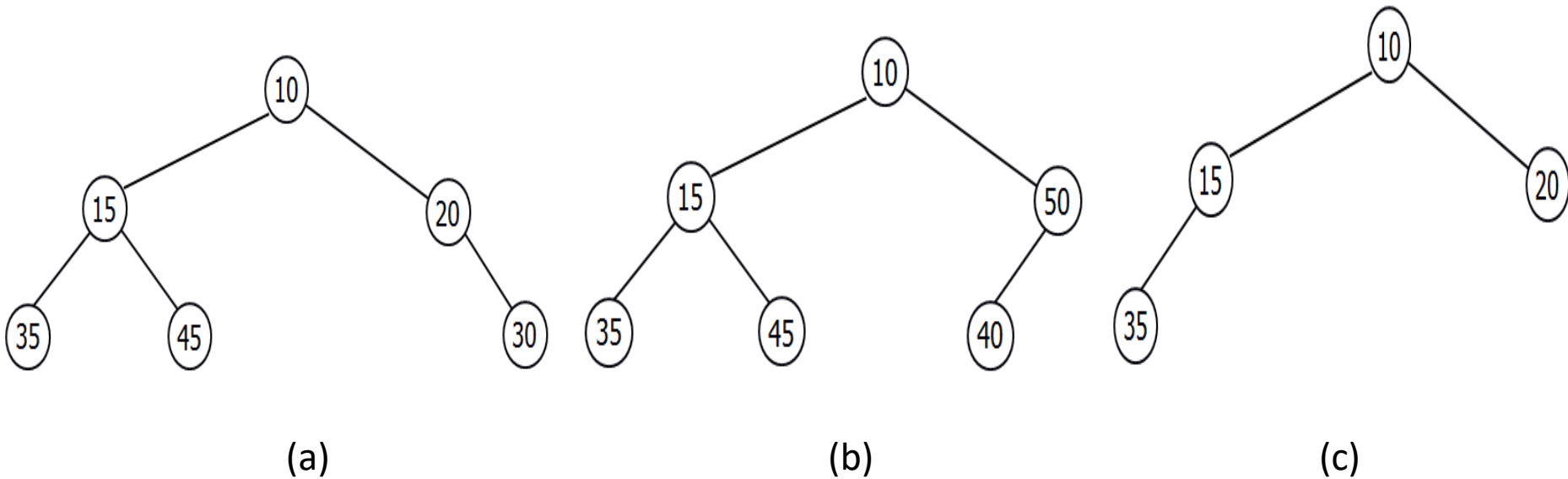
□ 스택과 큐와 같은 우선순위큐가 있는데, 왜 또 다른 우선순위큐 자료구조가 필요할까?

- 스택에 삽입되는 항목의 우선순위는 스택에 있는 모든 항목들의 우선순위보다 높음
- 큐에 삽입되는 항목의 우선순위는 큐에 있는 모든 항목들의 우선순위보다 낮음
- 삽입되는 항목이 임의의 우선순위를 가지면 스택이나 큐는 새 항목이 삽입될 때마다 저장되어 있는 항목들을 우선순위에 따라 정렬해야 하는 문제점이 있음

이진힙

[정의] 이진힙(Binary Heap)은 완전이진트리로서 부모의 우선순위가 자식의 우선순위보다 높은 자료구조

어느 트리가 이진힙일까?

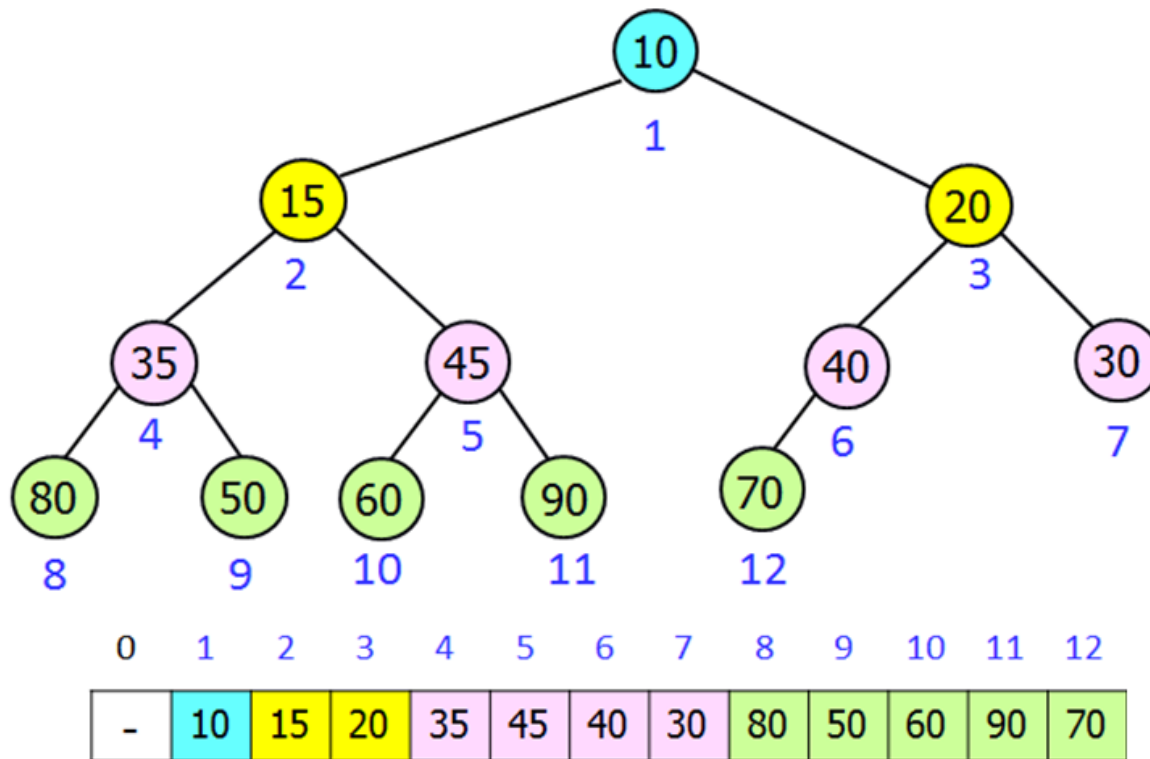


이진힙

- (a) 모든 노드들이 힙속성을 만족하지만 완전이진트리가 아님
- (b) 루트의 오른쪽 자식인 50이 40을 자식으로 가지고 있기 때문에 힙속성에 위배
- (c) 이진힙
 - 완전이진트리는 1차원 리스트로 구현하며, 리스트의 2 번째 원소부터 사용. 리스트 a에서 a[0]은 사용하지 않음
 - 완전이진트리의 노드들을 레벨순회(Level order Traversal) 순서에 따라 a[1]부터 차례로 저장

이진힙

완전이진트리의 노드들이 저장된 리스트



각 노드 아래의 숫자: 노드가 저장된 리스트 원소의 인덱스

이진힙

노드들을 리스트에 이와 같이 저장했을 때
힙에서 부모와 자식 관계

- $a[i]$ 의 자식은 $a[2i]$ 와 $a[2i+1]$ 에 있고,
 - $a[j]$ 의 부모는 $a[j/2]$ 에 있다, $j > 1$.
-
- 노드 35의 자식은 $a[2 \times 4] = a[8]$ 과 $a[2 \times 4 + 1] = a[9]$, 즉, 80과 50
 - 노드 90의 부모는 $a[11/2] = a[5]$ 의 노드 45

이진힙

- 이진힙의 종류:
 - 최소힙(Minimum Heap): 키 값이 작을수록 높은 우선순위
 - 최대힙(Maximum Heap): 키 값이 클수록 더 높은 우선순위
- 최소힙의 루트에는 항상 가장 작은 키가 저장됨
 - 부모에 저장된 키가 자식의 키보다 작다는 규칙 때문
 - 루트는 $a[1]$ 에 있으므로, $O(1)$ 시간에 min 키를 가진 노드 접근

힙 : 최소값 삭제(delete_min)

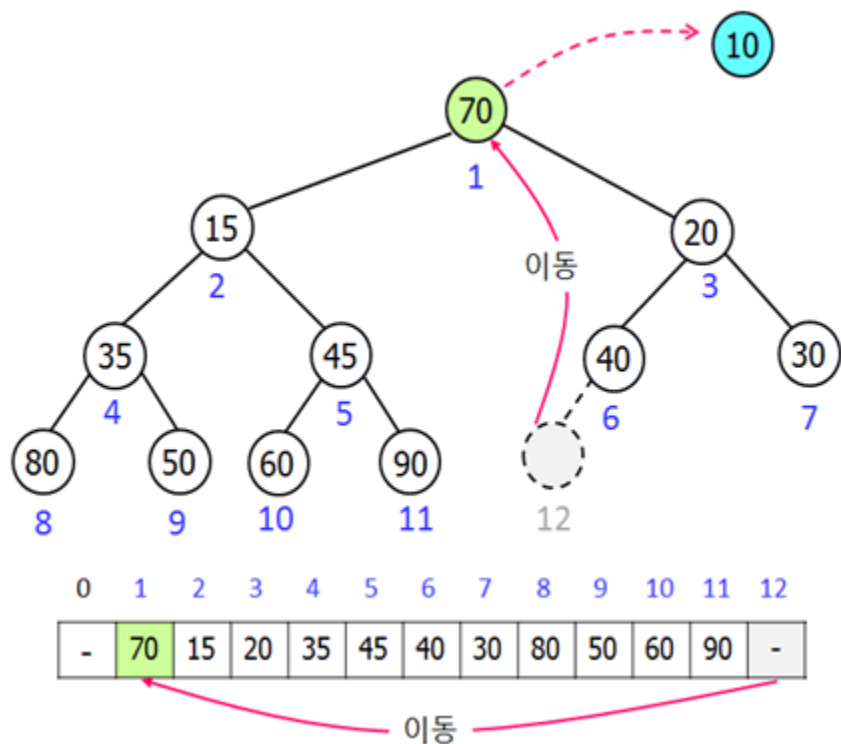
□ 루트의 키를 삭제

- [1] 힙의 가장 마지막 노드, 즉, 리스트의 가장 마지막 항목을 루트로 옮기고,
- [2] 힙 크기를 1 감소시킨다.
- [3] 루트로부터 자식들 중에서 작은 값을 가진 자식 (승자)과 키를 비교하여 힙속성이 만족될 때까지 키를 교환하며 이파리 방향으로 진행

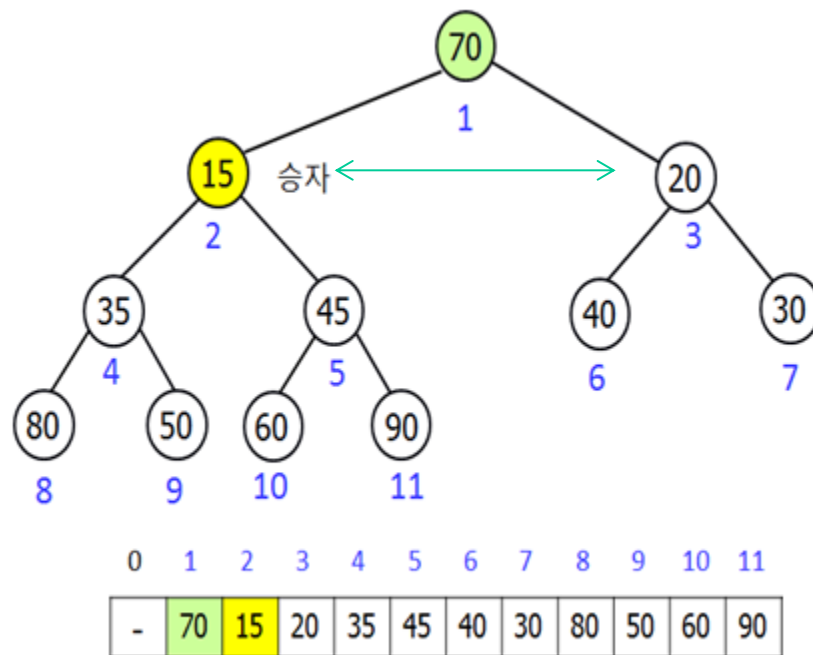
□ [3]의 과정은 루트로부터 아래로 내려가며 진행되므로 **downheap**이라 부름

힙 : 최소값 삭제(delete_min)

최소값 삭제 과정

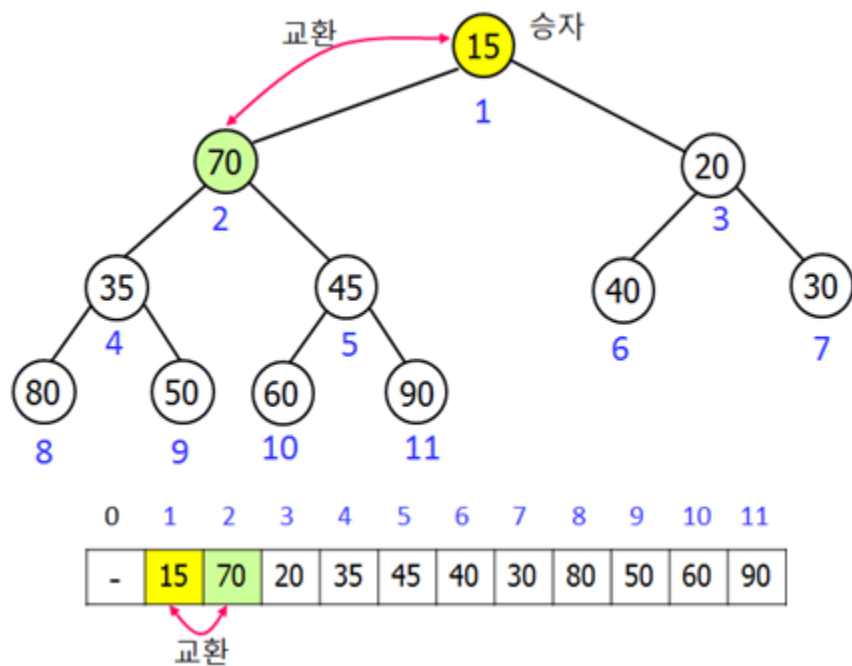


(a) 마지막 항목을 루트로 이동

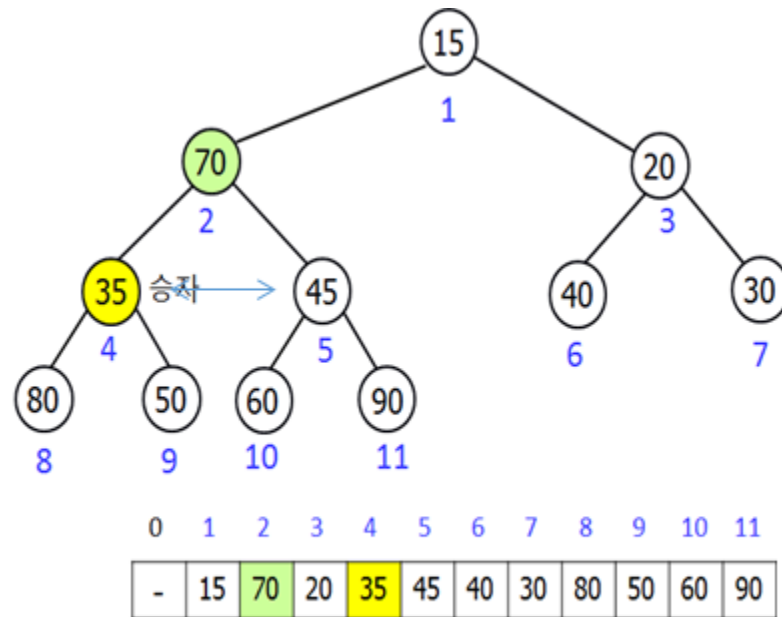


(b) 15와 20 중에 15가 승자

힙 : 최소값 삭제(delete_min)

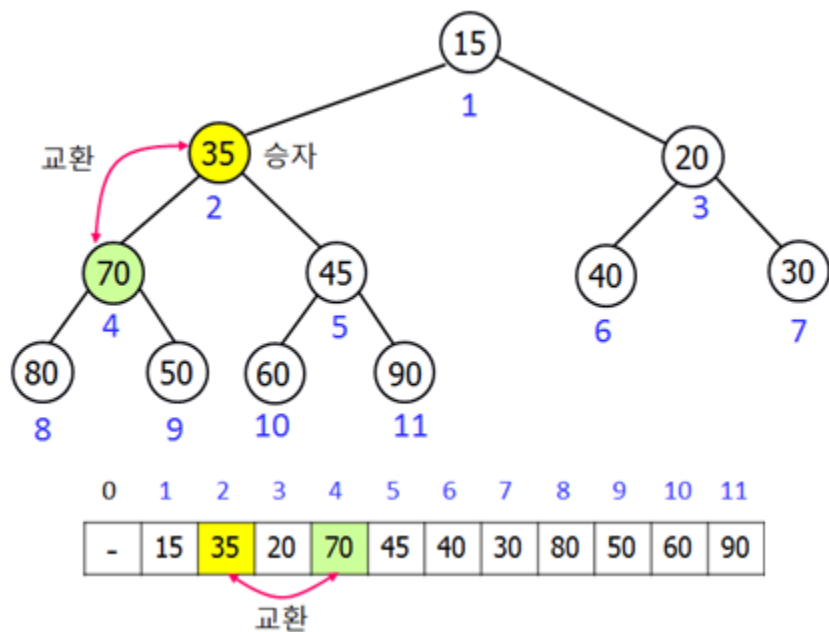


(c) 승자인 15와 루트를 교환

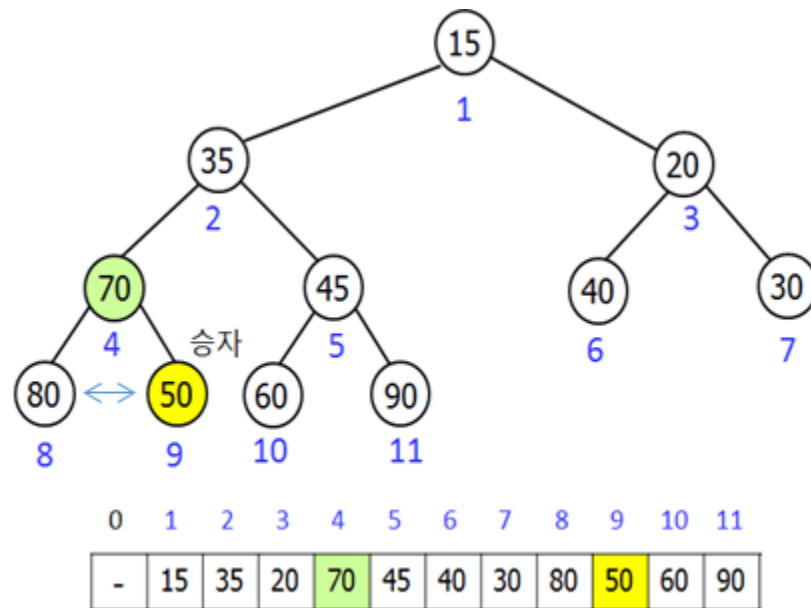


(d) 35와 45 중에 35가 승자

힙 : 최소값 삭제(delete_min)

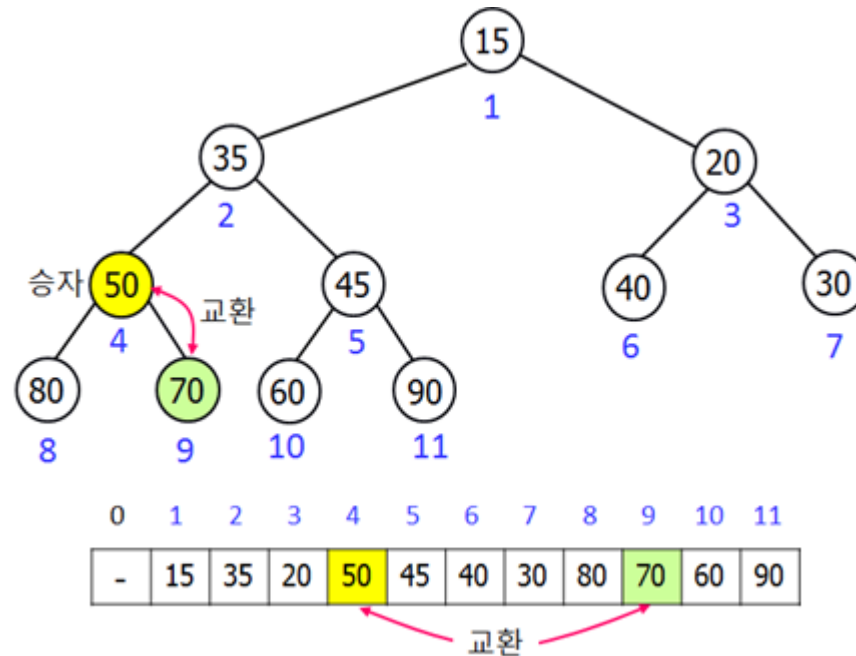


(e) 승자인 35와 70를 교환



(f) 80과 50 중에 50이 승자

힙 : 최소값 삭제(delete_min)



(g) 승자인 50과 70을 교환

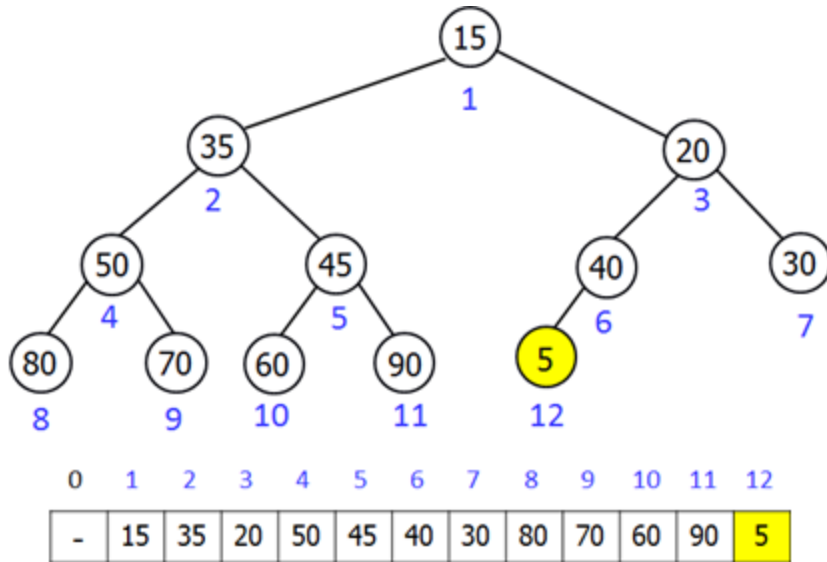
힙 : 삽입 연산(insert)

- [1] 힙의 마지막 노드(즉, 리스트의 마지막 항목)의 바로 다음 비어있는 원소에 새로운 항목을 저장
- [2] 루트 방향으로 올라가면서 부모의 키와 비교하여 힙속성이 만족될 때까지 노드를 교환

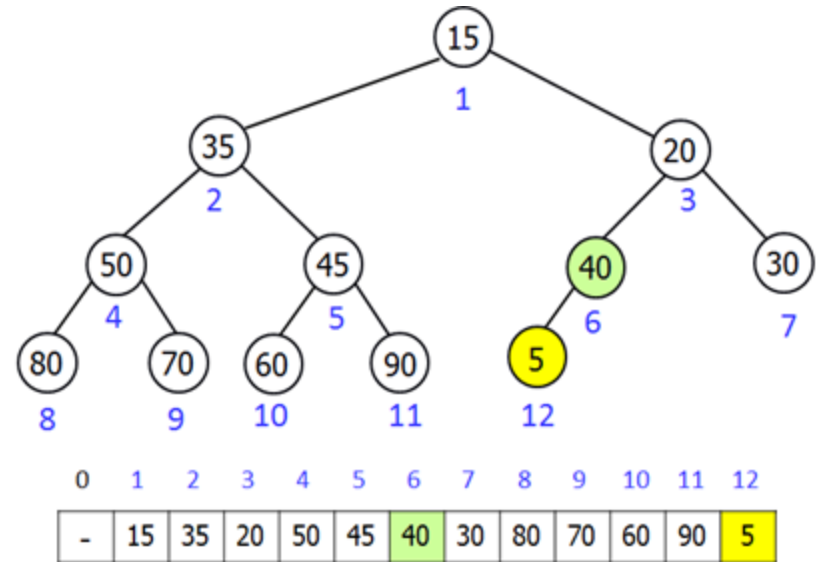
□ [2]의 과정은 이파리로부터 위로 올라가며 수행되므로 **upheap**이라 부름

힙 : 삽입 연산(insert)

최소힙에 5를 삽입하는 과정

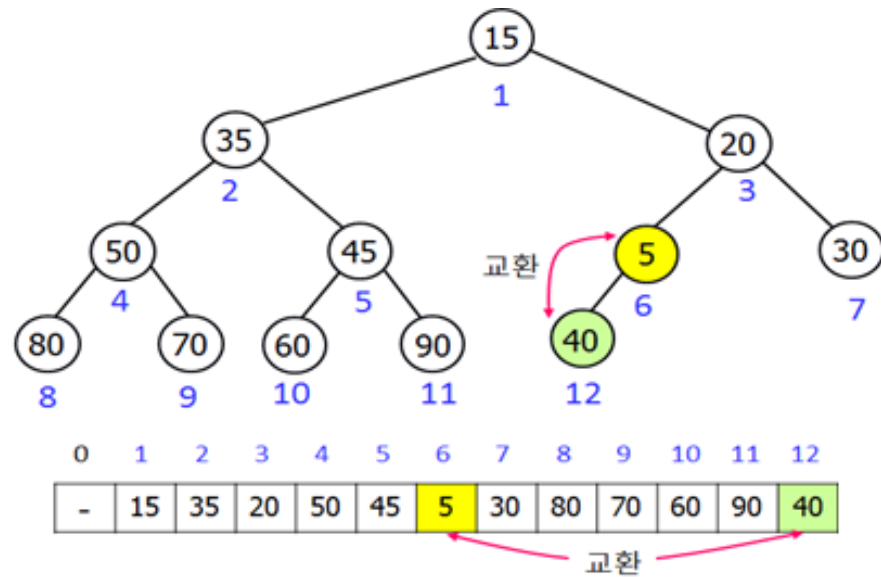


(a) 5를 마지막 항목(90) 다음에 저장

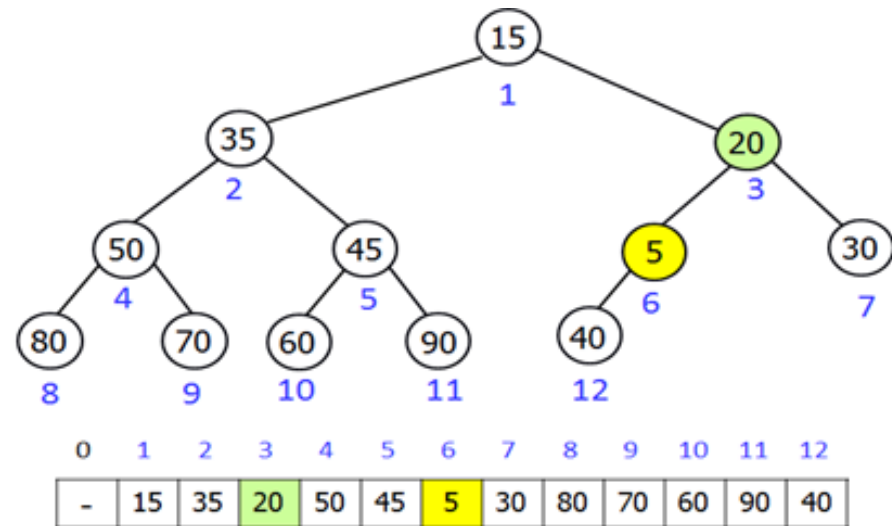


(b) a[12]의 5와 부모 a[6]의 40 비교

힙 : 삽입 연산(insert)

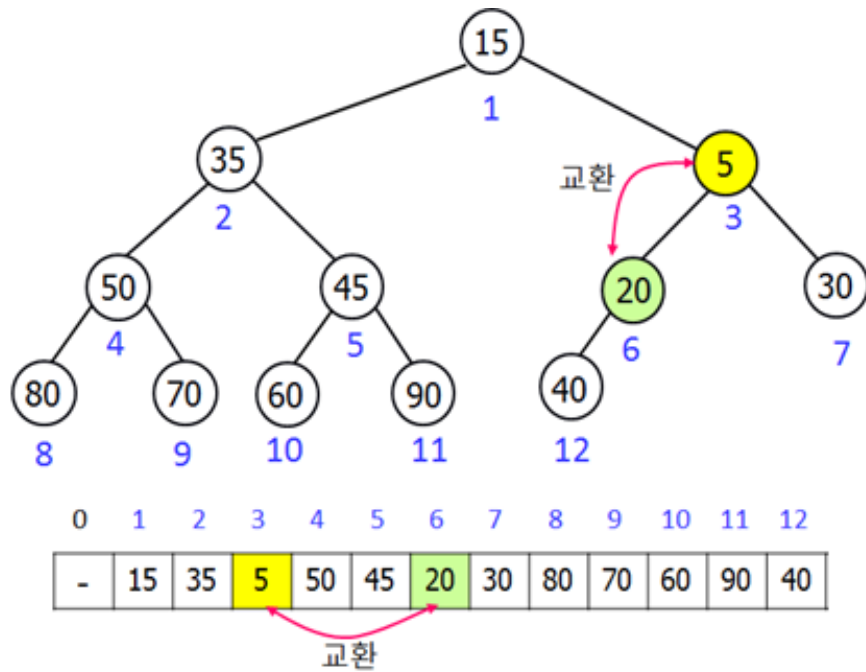


(c) 5와 40을 교환

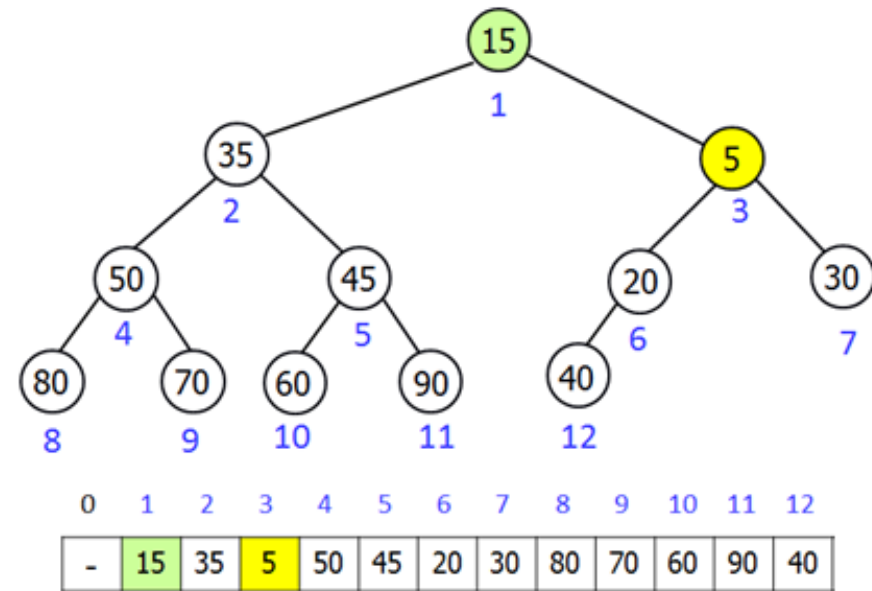


(d) a[6]의 5와 부모 a[3]의 20 비교

힙 : 삽입 연산(insert)

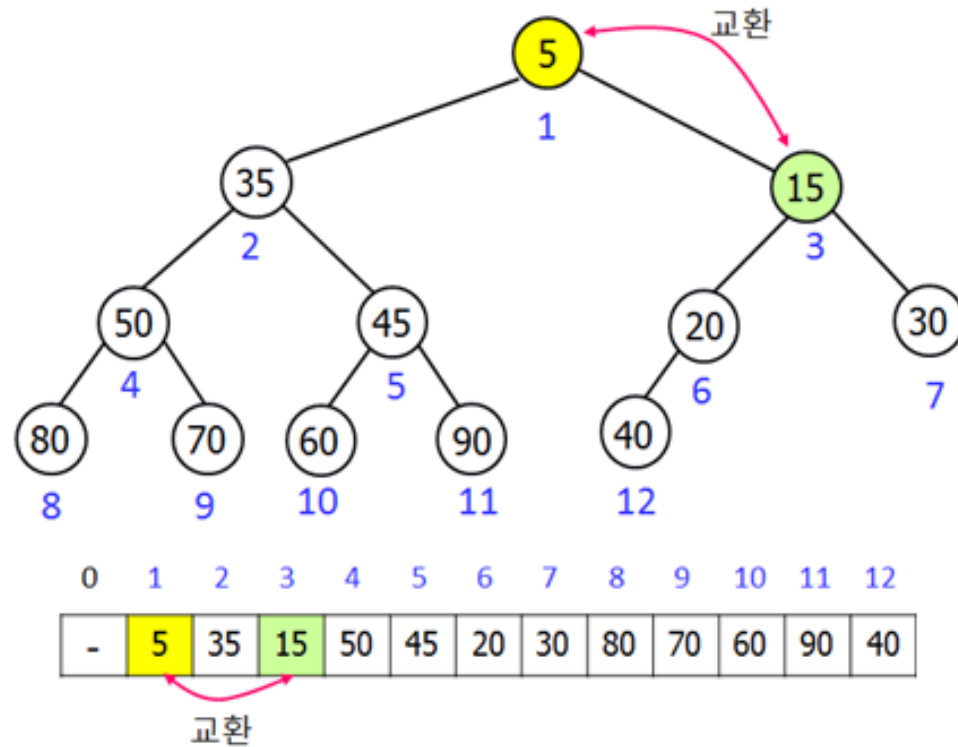


(e) 5와 20을 교환



(f) a[3]의 5와 부모 a[1]의 15 비교

힙 : 삽입 연산(insert)



(g) 5와 15를 교환

힙 정리

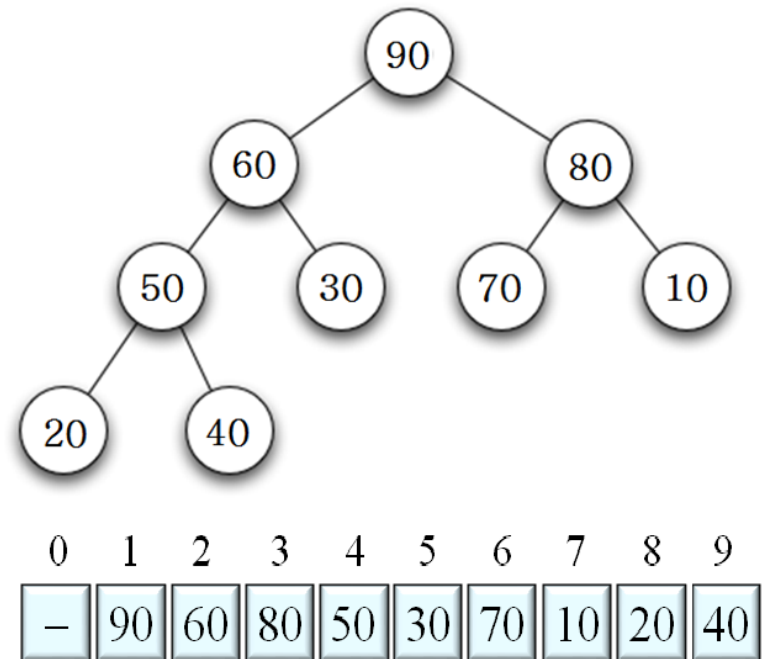
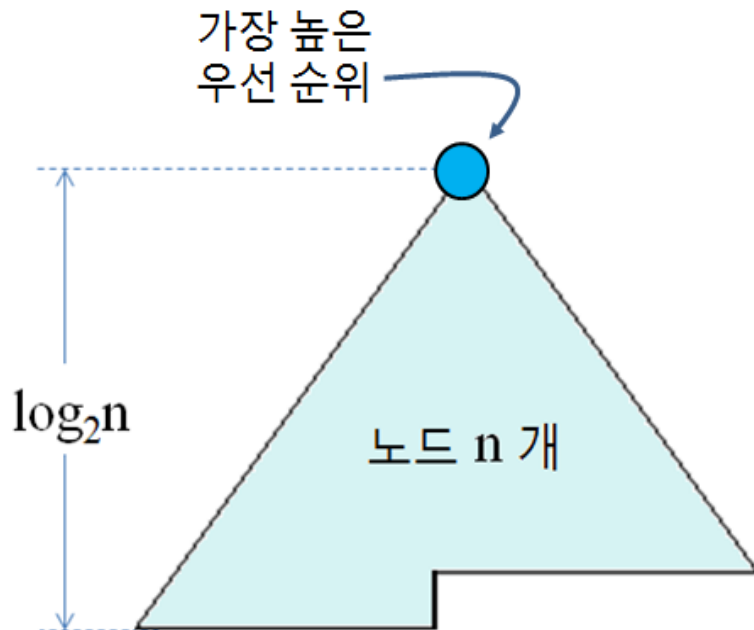
□ 힙 (heap)

- 힙 조건을 만족하는 완전 이진트리 (Complete Binary Tree)이다.
- 힙 조건이란 각 노드의 값이 자식 노드의 값보다 커야 한다는 것을 말한다.
- 노드의 값은 우선순위 (priority)라고 일컫는다.
- 힙의 루트에는 가장 높은 우선순위 (가장 큰 값)가 저장되어 있다.
- 값이 작을수록 우선순위가 높은 경우에는 가장 작은 값이 루트에 저장된다.

힙 정리

□ n개의 노드를 가진 힙

- 완전 이진트리이므로, 힙의 높이가 $\log_2 n$ 이며, 노드들을 빈 공간 없이 배열에 저장할 수 있다.
- 아래의 그림은 힙의 노드들이 배열에 저장된 모습



힙 정리

- 배열 A 에 힙을 저장한다면, $A[0]$ 은 비워 두고, $A[1] \sim A[n]$ 까지에 힙 노드들을 트리의 층별로 좌우로 저장한다.
 - 루트의 90이 $A[1]$ 에 저장되고,
 - 그 다음 층의 60과 80이 각각 $A[2]$ 와 $A[3]$ 에 저장되며,
 - 그 다음 층의 50, 30, 70, 10이 $A[4]$ 에서 $A[7]$ 에 각각 저장되고,
 - 마지막으로 20과 40이 $A[8]$ 과 $A[9]$ 에 저장된다.

힙 정리

□ 힙에서 부모 노드와 자식노드의 관계

- $A[i]$ 의 부모 노드 = $A[i/2]$
 - 단, i 가 홀수일 때, $i/2$ 에서 정수 부분만을 취한다. 예를 들어, $A[7]$ 의 부모 노드는 $A[7/2] = A[3]$ 이다.
- $A[i]$ 의 왼쪽 자식 노드 = $A[2i]$
 - $A[4]$ 의 왼쪽 자식 노드는 $A[2i] = A[2 \times 4] = A[8]$
- $A[i]$ 의 오른쪽 자식 노드 = $A[2i+1]$
 - 오른쪽 자식 노드는 $A[2i+1] = A[2 \times 4 + 1] = A[9]$

힙 정렬

□ 힙 정렬(Heap Sort)

- 힙 자료 구조를 이용하는 정렬 알고리즘이다.
- 오름차순의 정렬을 위해 최대힙(maximum heap) 구성한다.
- 힙의 루트에는 가장 큰 수가 저장되므로, 루트의 숫자를 힙의 가장 마지막 노드에 있는 숫자와 바꾼다.
- 즉, 가장 큰 수를 배열의 가장 끝으로 이동시킨 것이다.
- 그리고 루트에 새로 저장된 숫자로 인해 위배된 힙 조건을 해결하여 힙 조건을 만족시키고, 힙 크기를 1개 줄인다.
- 그리고 이 과정을 반복하여 나머지 숫자를 정렬한다.

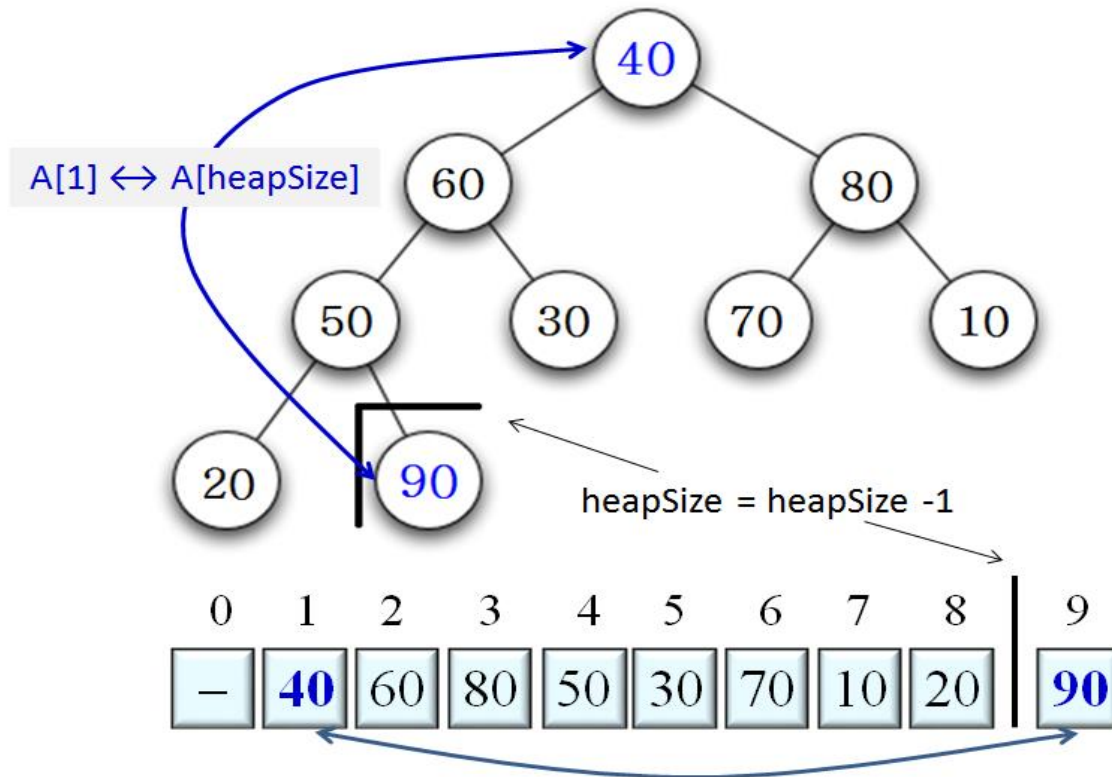
HeapSort 알고리즘

입력: 입력이 $A[1]$ 부터 $A[n]$ 까지 저장된 배열 A

출력: 정렬된 배열 A

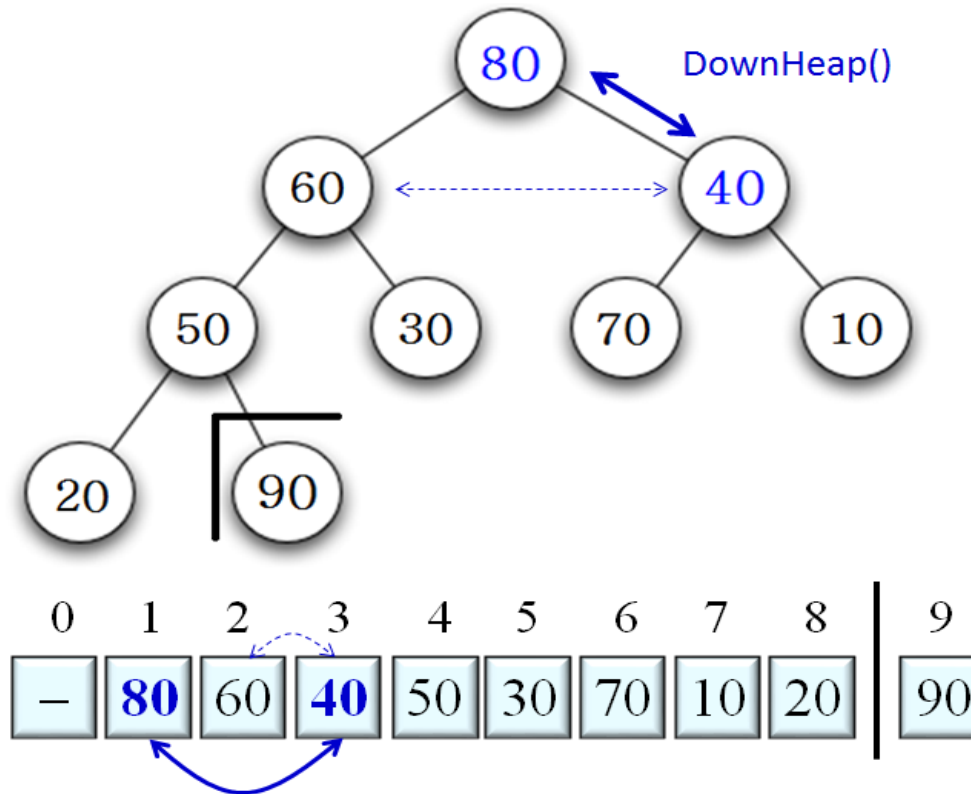
1. 배열 A 의 숫자에 대해서 힙 자료 구조를 만든다.
2. $\text{heapSize} = n$ // 힙의 크기를 조절하는 변수
3. for $i = 1$ to $n-1$
4. $A[1] \leftrightarrow A[\text{heapSize}]$ // 루트와 힙의 마지막 노드 교환
5. $\text{heapSize} = \text{heapSize} - 1$ // 힙의 크기를 1 감소
6. $\text{DownHeap}()$ // 위배된 힙 조건을 만족시킨다.
7. return 배열 A

HeapSort 알고리즘 수행 과정



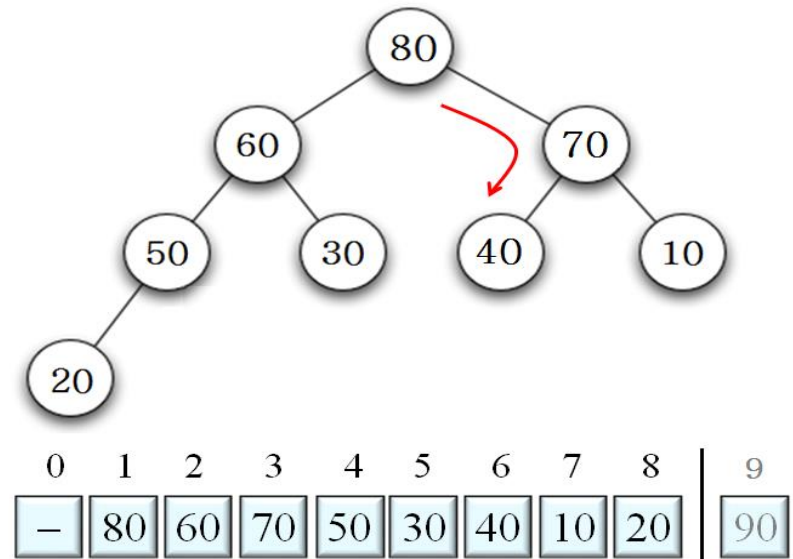
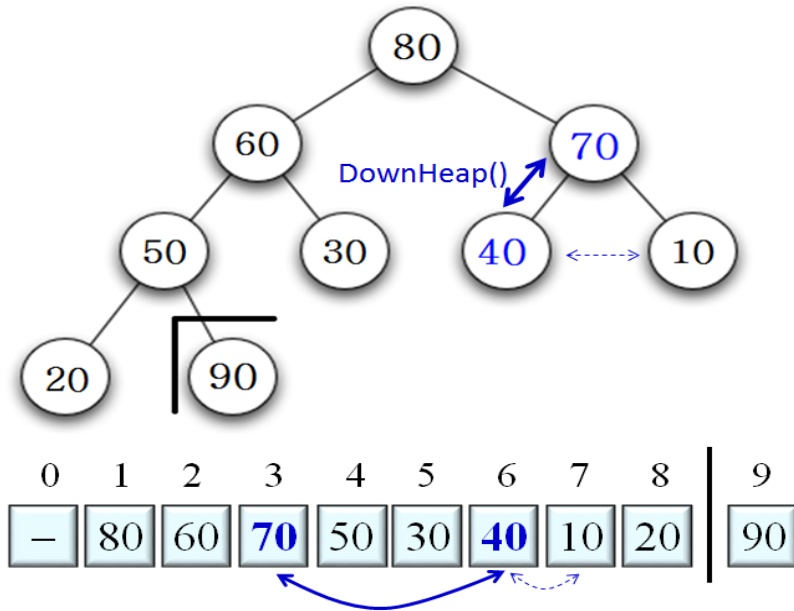
- Line 4에서 힙의 마지막 노드 40과 루트 90을 바꾸고, 힙의 노드 수(heapsize)가 1개 줄어든 것을 보이고 있다.

HeapSort 알고리즘 수행 과정



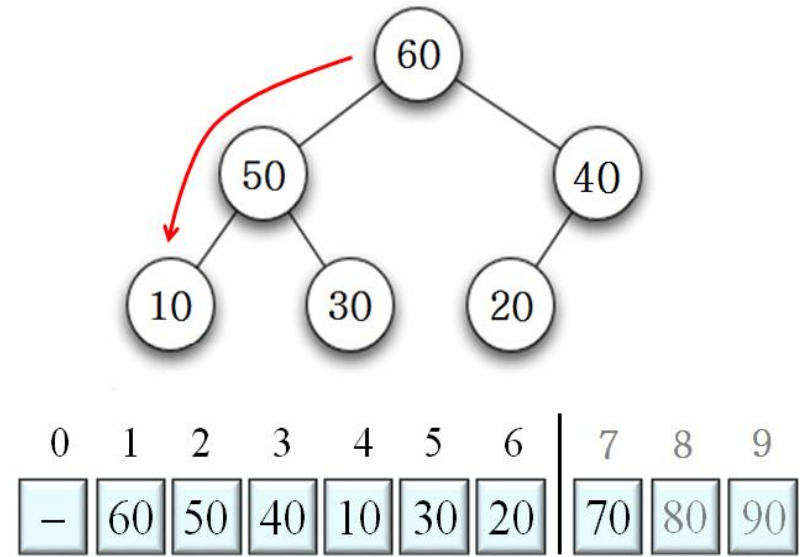
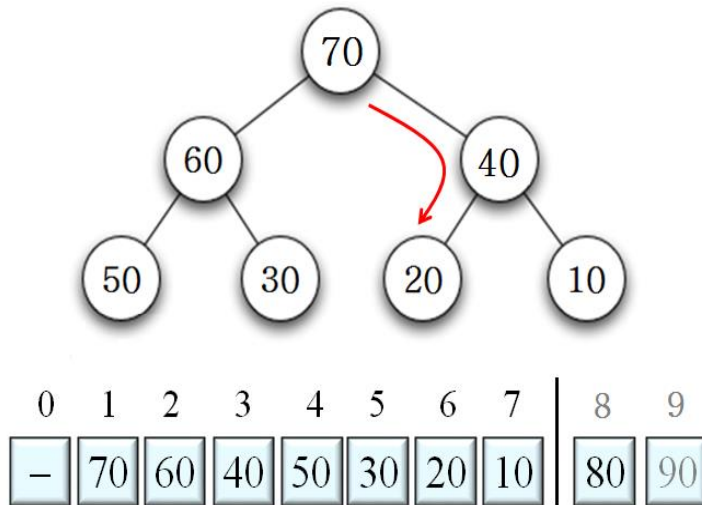
- 새로이 루트에 저장된 40이 루트의 자식 노드들 (60과 80)보다 작아서 힙 조건이 위배되므로 자식 노드들 중에서 큰 자식 노드 80과 루트 40이 교환된 것을 보여준다.

HeapSort 알고리즘 수행 과정



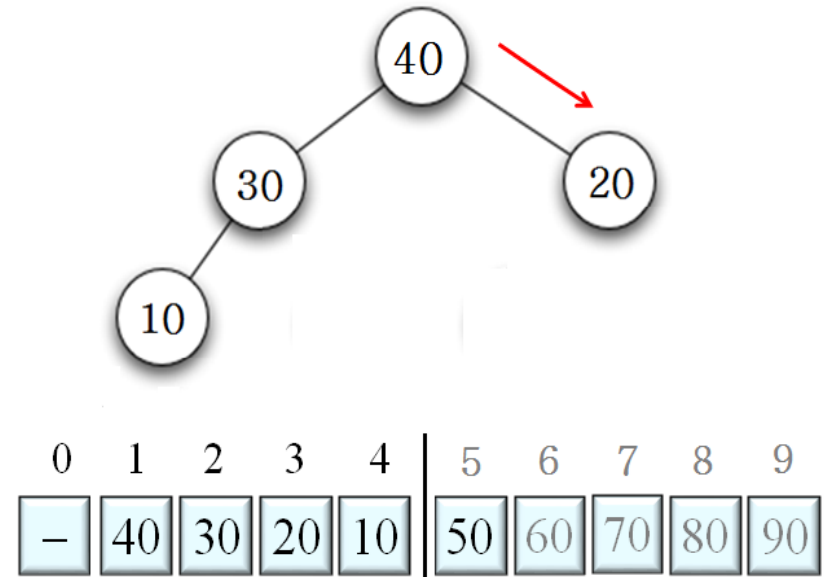
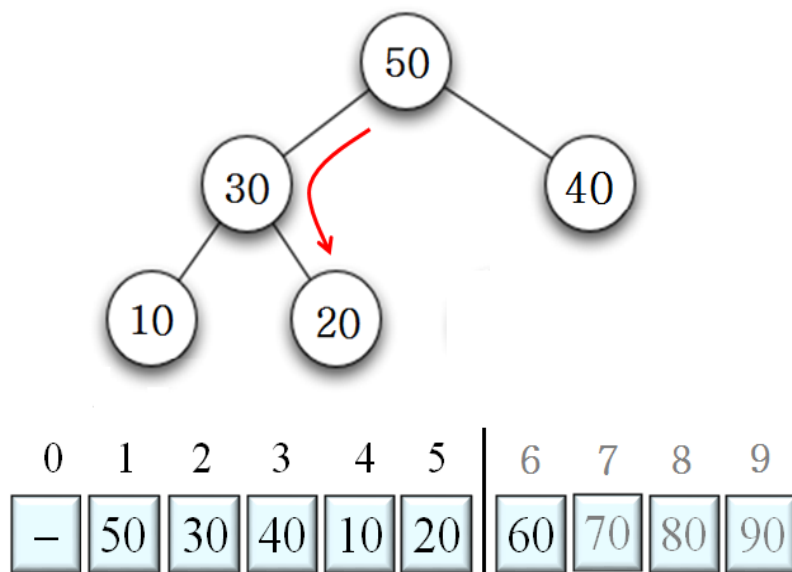
- ❑ 40은 다시 자식 노드들 (70과 10) 중에서 큰 자식 70과 비교하여, 힙 조건이 위배되므로 70과 40을 서로 바꾼다.
- ❑ 그 다음엔 더 이상 자식 노드가 없으므로 힙 조건이 만족되므로 DownHeap을 종료한다.

HeapSort 알고리즘 수행 과정



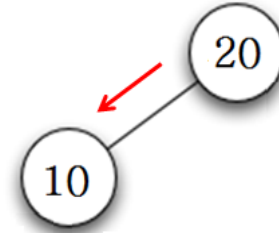
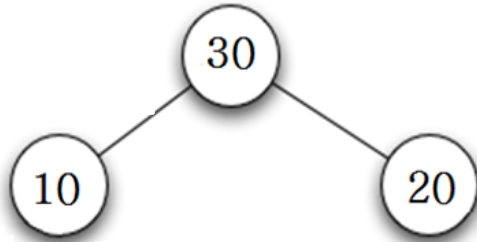
- ❑ 80이 20과 교환된 후 DownHeap을 수행한 결과 (왼쪽)
- ❑ 70이 10과 교환된 후 DownHeap을 수행한 결과 (오른쪽)

HeapSort 알고리즘 수행 과정



- ❑ 60이 20과 교환된 후 DownHeap을 수행한 결과 (왼쪽)
- ❑ 50이 20과 교환된 후 DownHeap을 수행한 결과 (오른쪽)

HeapSort 알고리즘 수행 과정

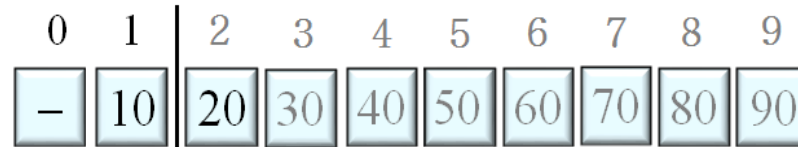


0	1	2	3	4	5	6	7	8	9
-	30	10	20	40	50	60	70	80	90

0	1	2	3	4	5	6	7	8	9
-	20	10	30	40	50	60	70	80	90

- 40이 10과 교환된 후 DownHeap을 수행한 결과 (왼쪽)
- 30이 20과 교환된 후 DownHeap을 수행한 결과 (오른쪽)

HeapSort 알고리즘 수행 과정



- ❑ 마지막으로 힙의 크기가 1이 되면 힙 정렬을 마친다.
- ❑ for-루프를 반복할 때마다 힙에서 가장 큰 수를 힙의 마지막 노드에 있는 수와 교환하고, 힙 크기를 1개 줄임으로써 힙에 속하지 않는 배열의 뒷부분에는 가장 큰 수부터 차례로 왼쪽 방향으로 저장된다.

시간복잡도

- ❑ Line 1: 힙을 만드는 데 $O(n)$ 시간
- ❑ Line 2: 변수를 초기화하므로 $O(1)$ 시간
- ❑ Line 3~6의 for-루프는 $(n-1)$ 번 수행되고, 루프 내부에서는 line 4~5가 각각 $O(1)$ 시간
- ❑ DownHeap은 최악의 경우 이파리 노드까지 내려가며 교환할 때이고, 힙의 높이는 $\log_2 n$ 이기 때문에 $O(\log n)$ 시간이 걸린다.
- ❑ 시간복잡도: $O(n) + O(n-1) \times O(\log n) = O(n \log n)$



Thank You